

Programming Basics

An introductory guide covering the core concepts every beginner programmer needs to know: what programming is, variables, data types, control structures, functions, basic object-oriented concepts, and how to handle errors.

1. What is Programming?

Programming is the process of giving a computer a set of precise instructions, written in a programming language, so that it performs a specific task. A program is simply an ordered list of such instructions. Computers cannot understand human language directly, so instructions are written in a programming language and then translated into machine code using a compiler or an

interpreter.

A compiler translates the entire source code into machine code before the program runs (examples: C, C++, Java). An interpreter translates and executes the code line by line while the program runs (examples: Python, JavaScript). Some languages, like Java, use a mix of both: the

source code is compiled into bytecode, which is then interpreted by a virtual machine.

Common programming languages include Python, Java, C#, C++, and JavaScript. Each language has its own syntax (the rules for writing valid code), but most languages share the same basic building blocks: variables, data types, operators, control structures, and functions.

2. Variables and Data Types

A variable is a named location in memory used to store a value that can change while the program runs. Before a variable can be used, it is usually declared, which means telling the program the variable's name and, in many languages, its data type.

The most common basic data types are:

- Integer (int): whole numbers, such as 10 or -5, with no decimal point.
- Floating-point (float/double): numbers with a decimal point, such as 3.14.
- String (str): a sequence of characters used to represent text, such as "Hello".

- Boolean (bool): a value that is either true or false, used for logical decisions.
- Character (char): a single letter, digit, or symbol.

Programming languages are either statically typed, where the data type of a variable must be known and fixed at compile time (such as Java and C#), or dynamically typed, where the type is determined automatically at runtime and can change (such as Python and JavaScript). A constant is similar to a variable but its value cannot be changed once it has been set.

Constants are useful for values that should never change during the program, such as the mathematical constant Pi.

3. Control Structures

Control structures determine the order in which the instructions of a program are executed.

Without them, a program would simply run every statement once, from top to bottom, with no decisions or repetition.

3.1 Conditional Statements

Conditional statements allow a program to make decisions. The most common form is the if statement, which runs a block of code only if a condition is true. It can be extended with else if (to

check another condition) and else (to run code when none of the previous conditions are true). A

switch statement is another way to choose between many possible cases based on the value of a single variable.

3.2 Loops

Loops allow a block of code to be executed repeatedly. The three most common loop types are:

- for loop: repeats a block of code a specific, known number of times.
- while loop: repeats a block of code as long as a condition remains true.
- do-while loop: similar to a while loop, but the code block runs at least once before the condition is checked.

The break statement immediately exits a loop, while the continue statement skips the rest of the current loop iteration and moves on to the next one.

4. Functions

A function (also called a method or procedure) is a named, reusable block of code designed to perform a specific task. Functions help avoid repeating the same code multiple times and make programs easier to read, test, and maintain.

A function is defined once and can be called (invoked) many times from different parts of a program. A function may accept input values, called parameters or arguments, and it may return a value back to the code that called it using a return statement. A function with no return statement, or one that returns nothing, is sometimes called a void function.

The scope of a variable defines the part of the program where that variable can be accessed. A variable declared inside a function is called a local variable and can only be used inside that function. A variable declared outside any function is called a global variable and can be accessed

from anywhere in the program.

5. Basic Object-Oriented Concepts

Object-Oriented Programming (OOP) is a programming approach based on the idea of "objects",

which bundle together data (attributes) and behavior (methods). Languages such as Java, C#, Python, and C++ support object-oriented programming.

The four main principles of OOP are:

- Encapsulation: bundling data and the methods that operate on that data together inside a single unit called a class, while restricting direct access to some of the object's internal details.
- Inheritance: allowing a new class (child/subclass) to reuse the attributes and methods of an existing class (parent/superclass).
- Polymorphism: allowing objects of different classes to be treated through a common interface, where each object responds to the same method call in its own way.
- Abstraction: hiding complex implementation details and exposing only the essential features of an object.

A class is a blueprint or template that defines the attributes and methods an object will have. An

object is a specific instance created from a class, with its own actual values for the class's attributes. For example, "Car" can be a class, while "a red Toyota with 5000 km on it" is an object

created from that class.

A constructor is a special method that is automatically called when a new object is created. Its main purpose is to initialize the object's attributes with starting values.

6. Errors and Debugging

Errors that occur while writing or running a program generally fall into three categories:

- Syntax errors: occur when code does not follow the grammatical rules of the programming language, such as a missing semicolon or an unmatched bracket. These are usually caught before the program runs.
- Runtime errors: occur while the program is running, such as trying to divide a number by zero or accessing an array index that does not exist.
- Logical errors: occur when a program runs without crashing but produces an incorrect result, because the logic used to solve the problem was wrong.

Debugging is the process of finding and fixing errors (bugs) in a program. Common debugging techniques include reading error messages carefully, adding print statements to check the value of variables at different points, and using a debugger tool to pause the program and step through the code line by line.

Exception handling is a technique used to deal with runtime errors gracefully instead of letting the

program crash. Many languages provide a try-catch (or try-except) structure: code that might cause an error is placed inside a try block, and code that handles the error if it occurs is placed inside a catch (or except) block.